

פרק 4

מערכים ומבני נתונים

מערכים

In C++:

```
int compileTimeSizedNumArray[20];  
// Size is defined in compile time!!  
// Size is handled separately.  
// No out of bounds check  
// Resizing is handled separately.  
// No such creature in C#..
```

```
int* runTimeSizedNumArray = new int[someSize];  
// Size is handled separately.  
// No out of bounds check  
// Dealloc is handled separately.  
// Resizing is handled separately.  
// No such creature in C#..
```

```
class IntArray {  
    private int* m_RunTimeSizedArray;  
    private int m_Size;  
  
    IntArray(int i_Size){  
        m_Size = i_Size;  
        m_RunTimeSizedArray = new int[m_Size];  
    }  
  
    ~IntArray(){  
        delete[] m_RunTimeSizedArray; }  
};
```

```
// continue..  
int GetSize() { return m_Size; }  
  
int GetAt(int i_Idx) {  
    if(i_Idx >= m_Size) throw "Exception!!";  
    else return m_RunTimeSizedArray[i_Idx];  
}
```

```
IntArray runTimeSizedIntArrayObj(20);  
IntArray* runTimeSizedIntArrayObj = new IntArray(20);  
// Size is handled internally. Out-of-Bounds check. Dealloc is handled internally  
// A dedicated class is needed for each type
```

מערכים - המשך

```
class Array<T> {
    private T* m_RunTimeSizedArray;
    private int m_Size;

    Array<T>(int i_Size){
        m_Size = i_Size;
        m_RunTimeSizedArray = new T[m_Size];
    }

    ~IntArray(){
        delete[] m_RunTimeSizedArray; }
}
```

```
// continue..
int GetSize() { return m_Size; }

T GetAt(int i_Idx) {
    if(i_Idx >= m_Size) throw "Exception!!!";
    else return m_RunTimeSizedArray[i_Idx];
}
}
```

```
Array<int> runTimeSizedIntArrayObj(20);
Array<float>* runTimeSizedFloatArrayObj = new Array<float>(20);
// Size is handled internally. Out-of-Bounds check. Dealloc is handled internally
```

```
// syntax in C#:
float[] runTimeSizedfloatArrayObj = new float[20];
// (classes are always referenced objects..)
// No resizing
```

```
// More dimensions in C#:
float[,] runTimeSizedfloatMatrixObj = new float[2,3];
runTimeSizedfloatMatrixObj[1,2] = someFloat;
// implemented as one buffer (no memory fragmentation!)
// No resizing
```

```
// Resizable array in C#:
List<float> resizableGenreicArray = new List<float>(20);
// Double re-allocation algorithm upon resizableGenreicArray.Add(someFloat);
```

תחביר מערכים ב- C#

מערכים חד מימדיים

הסוגריים המרובעים צמודים לטיפוס ולא לשם המשתנה.

אין מערך סטטי. המערך תמיד מוקצה דינאמית:

```
int[] arr = new int[5];  
arr[0] = 3;
```

אפשר להקצות במועד מאוחר יותר מאשר בעת ההכרזה:

```
DateTime[] arr;  
arr = new DateTime[12];
```

ניתן לאתחל ע"י רשימת ערכים:

```
int[] nums = new int[3]{1,2,3};  
string[] names = new string[]{"Tom", "Keren", "Gil"};  
int[] arr3 = {1,2,3};
```

האינדקס במערכים הוא על בסיס 0 (zero-based arrays)

הגישה למערך נבדקת בזמן ריצה וחריגות יזרקו במקרה של חריגה מגבולות

מערכים - המשך

מערכים רב מימדיים (לא קיים ב-Java) 

Rectangular arrays – מערכים מלבניים 

```
int[,] arr = new int[,]{{1,2,3},{4,3,2}};  
// or:  
int[,] arr = new int[2,3];  
arr[0,0] = 1;  
arr[0,1] = 2;  
arr[0,2] = 3;  
arr[1,0] = 4;  
arr[1,1] = 3;  
arr[1,2] = 2;
```

מספר רב יותר של מימדים: 

```
int[,,,] arr = new int[2,3,7,2];  
arr[0,1,4,1] = 1;
```

מקום רצוף בזיכרון 

מערכים - המשך

מערכים רב מימדיים

Jagged Arrays – "משוננים"

מערך של מערכים

```
int[][] arr = new int[3][];  
arr[0] = new int[1];  
arr[1] = new int[2];  
arr[2] = new int[]{1,2,3};
```

המערך כאובייקט

arr.Length

```
for(int i = 0 ; i < arr.Length ; ++i)  
{...}
```

כל המערכים יורשים מ- Array – מאפשר פולימורפיזם

מתודות סטטיות (למשל: Array.Sort(arr))

רשימות, אוספים ומבני נתונים אחרים

■ מבני נתונים פולימורפיים לפני C# 2.0:

System.Collection.ArrayList ■

System.Collection.Hashtable ■

System.Collection.Stack ■

■ בשונה ממערכים:

■ גודל האוסף דינאמי (להבדיל ממערך שבו הגודל קבוע)

■ אפשרות להגנה על האוסף מפני כתיבה (readonly)

■ חסרונות לעומת מערכים:

■ ביצועים (type-casting / boxing)

■ פשטות קוד (אין המרה)

```
int capacity = 50;
ArrayList students = new ArrayList(capacity);
students.Add(new Student()); // add a new student to the list
students.RemoveAt(3); // remove the 4th student in the list
Student s = (Student)students[2]; // get the 3rd student in the list
```

רשימות, אוספים ומבני נתונים אחרים

מבני נתונים גנריים החל מ- C# 2.0

System.Collection.Generic.List<T>

System.Collections.Generic	System.Collections
Comparer<T>	Comparer
Dictionary<K,T>	HashTable
LinkedList<T>	-
List<T>	ArrayList
Queue<T>	Queue
SortedDictionary<K,T>	SortedList
Stack<T>	Stack
ICollection<T>	ICollection
IComparable<T>	System.IComparable
IDictionary<K,T>	IDictionary
IEnumerable<T>	IEnumerable
IEnumerator<T>	IEnumerator
IList<T>	IList

```
List<Student> students = new List<Student>();  
students.Add(new Student()); // add a new student to the list  
students.RemoveAt(3); // remove the 4th student in the list  
Student s = students[2]; // get the 3rd student in the list (no casting!!)
```


סריקת אוספים

במקום:

```
1. for(int i = 0 ; i < computers.Count ; i++)  
2. {  
3.     computers[i].Restart();  
4. }
```

foreach

ריצה על מערכים/אוספים/טיפוס שמממש IEnumerator

אסור לבצע שינויים על האוסף שעליו רצים

```
1. foreach(Computer comp in computers)  
2. {  
3.     comp.Restart();  
4. }
```